

“Event and Room Occupancy Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Duddukuri Sri Tanmai **2520080042**

Sahasra **2520080062**

Valaboju Akshitha **2520080003**

Under the Esteemed Guidance of

Khaja Shareef

CSIT HOD

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

We hereby declare that the project entitled “Sign Language Translator App – SignEase” is our original work carried out as part of the academic requirements of our course.

We declare that this project has been developed by us under the guidance of our project guide. The work presented in this report is based on our own efforts, implementation, and learning. We further declare that this project has not been submitted, either fully or partially, to any other institution or university for the award of any degree or academic qualification.

We have made every effort to acknowledge the resources, documentation, and references used during the development of this project.

D.Sri Tanmai - 2520080042

Sahasra -2520080062

V.Akshitha - 2520080003

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**Sign Language Translator App – SignEase**” is a bonafide work done and submitted by **[Tanmai] – [2520080042]** , **[Sahasra] – [2520090042]** and **[Akshitha] – [2520080003]** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, **Section 12, Team 22, at KLH University** during **2026–2027, Trimester 4**.
The project has been carried out under the guidance and supervision of the undersigned and is submitted in partial fulfillment of the academic requirements of the course.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of the project “**Sign Language Translator App – SignEase**” would not have been possible without the valuable support, guidance, and encouragement of many people. We would like to express our sincere gratitude to everyone who assisted us directly or indirectly in the completion of this project.

We express our deepest appreciation to our **Course Coordinator, Dr. Khaja Shareef**, and our project guide, **Divya Priya Degala**, Department of Computer Science and Information Technology, for their tremendous support, encouragement, valuable suggestions, and guidance throughout the development of this project. We express our gratitude to **Dr. Kaja Shareef**, Head of the **Department of Computer Science and Information Technology**, for providing us with the necessary facilities, resources, and support that enabled us to successfully complete this project-based lab.

We also extend our sincere thanks to all the **teaching and non-teaching staff members** of the department for their assistance and cooperation throughout the project. Finally, we sincerely thank our **parents, friends, and classmates** for their constant support, encouragement, and cooperation during the development and completion of our project.

Thank You.

Tanmai – 2520080042

Sahasra – 2520080062

V.Akshitha - 2520080003

Abstract

SignEase is a web-based Sign Language Translator application designed to support communication between people who use sign language and people who may not understand it. Communication barriers can make everyday interactions difficult, especially in educational, public, and social environments. SignEase aims to provide a simple and accessible interface through which users can explore sign-language gestures and convert selected signs into text and speech.

The application includes a user-friendly dashboard, a sign dictionary, a learning section, translation history, favorites, and a translator interface. The prototype uses HTML, CSS, and JavaScript to build the frontend. Browser-based features such as camera access and speech synthesis are used to support the demonstration. LocalStorage is used to maintain selected information in the browser.

The current prototype demonstrates the application workflow using predefined demo recognition rather than a trained machine-learning model. Future development can integrate computer vision and machine learning for gesture recognition, along with a backend and database for secure user accounts and persistent records. SignEase aims to make sign-language learning and communication support more accessible through a simple web application.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17

10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18
----	-------------------------------------	--	----

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Communication plays an important role in education, employment, social interaction, and everyday activities. People who communicate using sign language may experience difficulties when interacting with individuals who are unfamiliar with sign language. This communication gap can make simple interactions more difficult and may reduce accessibility in different environments.

Sign language communicates information through hand gestures, movements, facial expressions, and body language. Although sign language is an effective communication method, learning and understanding it requires practice and appropriate resources.

With the development of web technologies, applications can be created to provide accessible tools for learning and communication. A web-based application can allow users to explore signs, understand their meanings, and practise common gestures through an interactive interface.

SignEase – Sign Language Translator App is proposed as a web-based solution to support sign-language learning and translation demonstrations. The application provides multiple modules through a centralized interface and is designed to be simple and easy to use.

The current prototype demonstrates the frontend workflow of a sign-language translation system. It provides camera access and demo translation functionality while preparing the application architecture for future integration with computer vision and machine-learning technologies.

1.2 Problem Statement

People who use sign language can face communication barriers when communicating with individuals who do not understand sign language. In many situations, communication may depend on another person who understands the signs.

Learning sign language can also be difficult when users do not have access to organized learning resources. Existing resources may be distributed across different platforms, making it difficult for beginners to learn and practise common signs in one place.

The proposed SignEase application aims to provide a centralized and user-friendly platform for exploring sign-language gestures, displaying their meanings, and supporting text and speech output.

The long-term objective is to enable real-time sign recognition using a camera and machine-learning model.

1.3 Scope of the Project

The scope of SignEase includes:

- Development of a web-based sign-language application.
- User-friendly landing page and dashboard.
- Sign translation interface.
- Camera access through the browser.
- Demo sign-to-text translation.
- Text-to-speech output.
- Sign dictionary.
- Learning section.
- Translation history.
- Favorite signs.
- User profile interface.
- Feedback interface.
- Responsive frontend design.
- Preparation for future machine-learning integration.

The current project primarily focuses on the frontend prototype. A trained ML model, centralized backend, and production database can be integrated in future versions.

1.4 Objectives

The main objectives of the project are:

1. To develop a simple and user-friendly Sign Language Translator application.
 2. To provide an interface for demonstrating sign-to-text translation.
 3. To provide camera access for future real-time sign recognition.
 4. To convert translated text into speech.
 5. To provide a sign dictionary for learning and reference.
 6. To provide a learning module for beginners.
 7. To maintain translation history.
-

8. To allow users to save frequently used signs as favorites.
9. To develop an accessible and responsive web interface.
10. To create a foundation for future machine-learning and database integration.

1.5 Importance of the Application

SignEase is important because it addresses a communication and accessibility problem through technology. A digital application can provide users with easy access to sign-language-related information and learning resources.

The application can help users become familiar with common signs and understand how gestures can be represented as text and speech.

It also demonstrates how web technologies, browser APIs, and future machine-learning technologies can be combined to develop accessibility-focused applications.

1.6 Target Users / Audience

The main target users of SignEase include:

- People interested in learning sign language.
- People who communicate using sign language.
- Students.
- Teachers and educators.
- Individuals interacting with sign-language users.
- Accessibility-focused organizations.
- Developers and researchers interested in gesture recognition.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has relatively simple hardware requirements because the current version is a web-based frontend application.

Recommended hardware includes:

Laptop or desktop computer.

Minimum 4 GB RAM.

Dual-core processor or better.

Webcam or built-in camera for translator testing.

Speakers or headphones for speech output.

Keyboard and mouse/touchpad.

Sufficient storage for project files.

Internet connection for development resources and future integrations.

2.2 Software Requirements

The software requirements are:

- Windows, Linux, or macOS operating system.
- Visual Studio Code.
- Google Chrome, Microsoft Edge, or Mozilla Firefox.
- HTML5 support.
- CSS3 support.
- JavaScript support.
- Browser support for MediaDevices API.
- Browser support for Web Speech API.
- Live Server extension for convenient local development.

2.3 Development Tools and Frameworks

The project uses the following development tools and technologies:

- **Visual Studio Code:** Main code editor.
- **HTML5:** Application structure.
- **CSS3:** Application styling and responsive layout.
- **JavaScript:** Application logic and interaction.
- **MediaDevices API:** Camera access.
- **Web Speech API:** Text-to-speech functionality.
- **LocalStorage:** Browser-based storage for demo information.

- **Git/GitHub:** Can be used for version control and project collaboration.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend of SignEase is developed using HTML5, CSS3, and JavaScript.

HTML5 defines the structure of the application. CSS3 is used to create the visual design, layouts, buttons, cards, forms, and responsive interface. JavaScript provides interactive functionality and controls application behavior.

The frontend contains multiple sections including the landing page, authentication interface, dashboard, translator, dictionary, history, favorites, learning section, profile, and feedback

3.2 Backend

The current SignEase prototype does not contain a connected backend server.

The application performs its current demonstration functionality directly in the browser. Future development can introduce a backend using technologies such as Node.js and Express.js.

A backend can provide APIs for user authentication, sign management, history, favorites, feedback, and learning progress.

3.3 Database

The current prototype uses LocalStorage for selected demo information. LocalStorage allows information such as favorites and history to remain available within the same browser.

For a complete DBMS implementation, a centralized database can be added in future development.

Possible database options include:

- MySQL
- PostgreSQL
- MongoDB

A future database can contain entities such as Users, Signs, Categories, Translation History, Favorites, Lessons, User Progress, and Feedback.

3.4 Version Control

Git can be used as the version-control system for SignEase. Git allows developers to track changes to source code and maintain different versions of the project.

GitHub can be used to store the project repository and support collaboration between team members.

3.5 Authentication and Security Libraries

The current frontend prototype provides login and registration interfaces but does not implement complete backend authentication.

For a production version, authentication can be implemented using secure backend authentication methods.

Passwords should be stored using secure hashing algorithms such as bcrypt rather than plain text.

Authentication tokens such as JWT can be used for maintaining secure sessions.

User input should also be validated and sensitive information should never be exposed through frontend code.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The **Sign Language Translator App – SignEase** follows a layered architecture that separates the user interface, application logic, and data storage. This structure makes the application easier to develop, maintain, and extend with advanced technologies such as machine learning, computer vision, backend services, and a centralized database. The architecture consists of three main layers: **Presentation Layer, Application/Logic Layer, and Data Layer.**

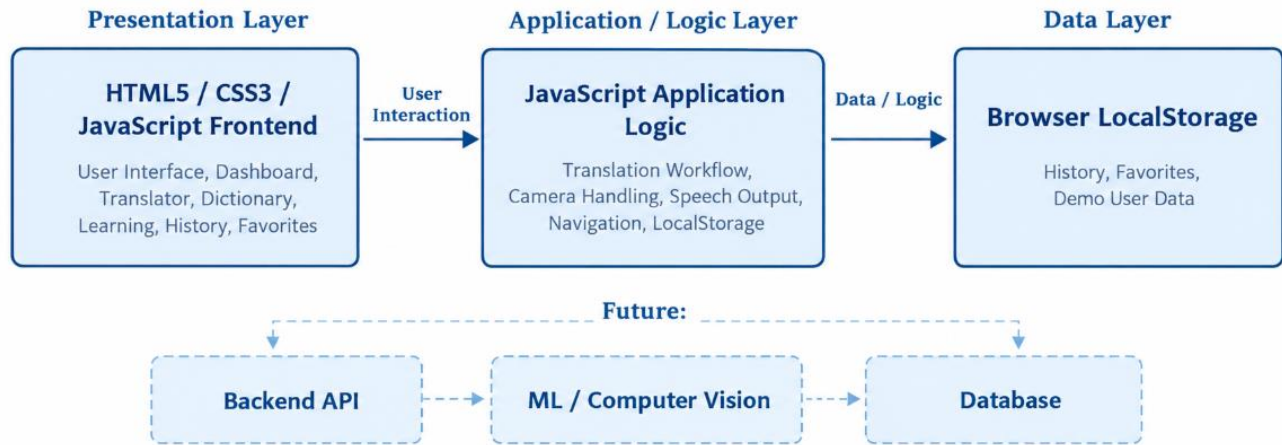


Figure 4.1: High-level architecture of the Sign Language Translator App – SignEase

4.2 Description of Each Layer

Presentation Layer

The presentation layer contains the user interface. It is developed using HTML and CSS and provides navigation and visual components for the user.

Application Logic Layer

JavaScript handles application interactions, navigation, translation demonstration, favorites, history, and communication with browser APIs.

Browser API Layer

The MediaDevices API provides camera access, while the Web Speech API provides speech synthesis.

Local Storage Layer

LocalStorage is used to maintain selected demo data such as history and favorites.

Future Backend Layer

A future backend can process API requests, manage authentication, communicate with the database, and connect the frontend with the ML model.

Future Database Layer

A database can provide persistent storage for users, signs, history, favorites, lessons, and other application data.

4.3 Deployment Architecture

The current prototype can run on a local computer using a modern web browser.

During development, Visual Studio Code and a local development server can be used to run the application.

In a future deployment, the frontend can be hosted on a web server or cloud platform, while the backend API and database can be deployed separately.

CHAPTER 5: DESIGN

5.1 Database Design Principles

Although the current prototype uses LocalStorage, a future database design for SignEase should follow standard DBMS principles.

The database should:

- Use primary keys for unique identification.
- Use foreign keys for relationships.
- Reduce duplicate information through normalization.
- Validate important data.
- Maintain referential integrity.
- Protect sensitive user information.
- Separate user information from application content.
- Maintain timestamps for important records.

5.2 ER Diagram / Database Schema

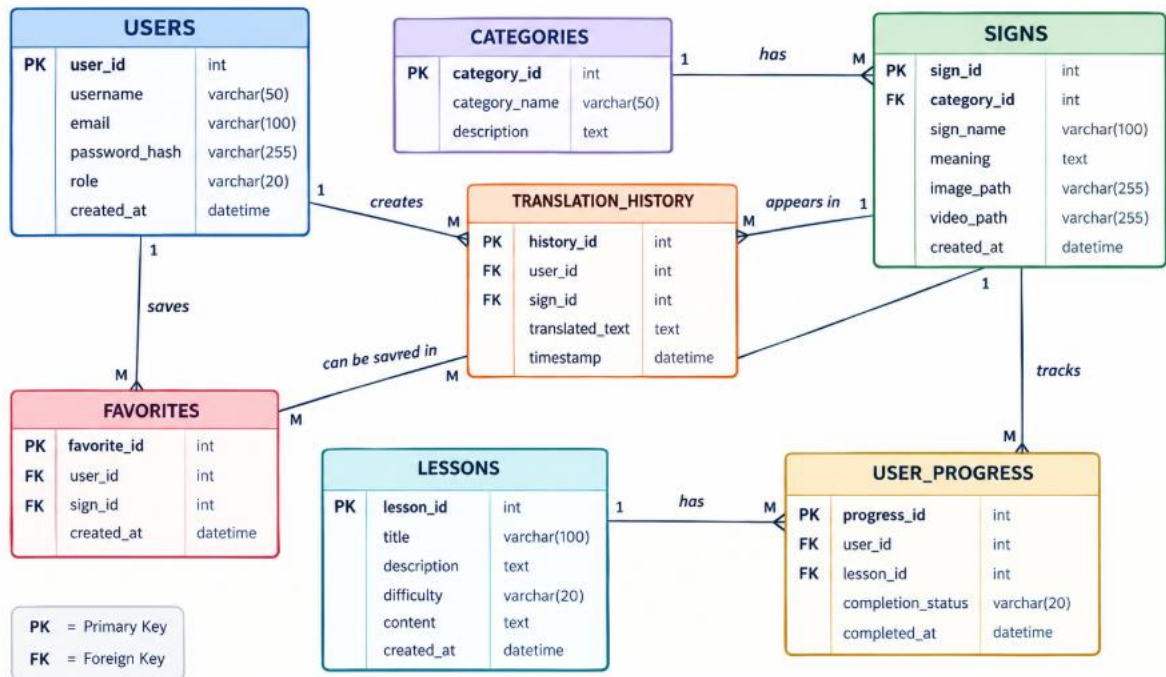


Figure 5.1: Entity-Relationship Diagram of the SignEase Database Schema

The database follows a relational structure in which each entity has a unique primary key. Foreign keys are used to establish relationships between related tables. For example, `category_id` in the `SIGNS` table connects each sign with its corresponding category, while `user_id` and `sign_id` in `TRANSLATION_HISTORY` identify the user and sign associated with a translation.

The `FAVORITES` table acts as a relationship table between users and signs, allowing multiple users to save the same sign while each user can maintain their own collection of favorite signs.

The `USER_PROGRESS` table connects users with lessons and allows the system to track learning activity.

This structure reduces unnecessary duplication of data and provides a foundation for implementing a centralized database in the future.

5.3 Design Decisions and Assumptions

The project is designed as a web application so that users can access it through a browser without installing a dedicated application.

HTML, CSS, and JavaScript were selected because they provide a simple and flexible platform for frontend development.

The current translator uses demo/predefined recognition rather than a trained ML model. This allows the frontend workflow to be demonstrated before implementing the computationally more complex recognition system.

LocalStorage is used for prototype-level data persistence. A centralized database will be required for multi-user access and permanent storage.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

Landing Page

The landing page introduces SignEase and provides information about the application's purpose and features.

Login and Registration

The authentication interface provides forms through which users can enter their account information. Complete server-side authentication is planned for a future version.

Dashboard

The dashboard acts as the main navigation area and provides access to different application modules.

Translator

The translator module provides a camera interface and translation output area. The current version demonstrates the translation workflow using predefined/demo recognition.

Sign Dictionary

The dictionary displays signs and their meanings so that users can explore and learn common gestures.

Learning Module

The learning module provides content for users who want to practise sign language.

History

The history section displays previous demo translation activities stored in LocalStorage.

Favorites

The favorites section provides access to signs saved by the user.

Profile

The profile section displays user-related information.

Feedback

The feedback section provides an interface for users to submit comments or suggestions.

6.2 Front-end Logic

The frontend logic is implemented using JavaScript.

JavaScript handles:

- Navigation.
- Button interactions.
- Form interactions.
- Demo translation.
- Camera initialization.
- Speech output.
- Favorites.
- Translation history.
- LocalStorage operations.
- Dynamic content updates.

CSS provides the visual styling and responsive layout.

6.3 API Integration

The current prototype does not use a custom REST API.

It uses browser-provided APIs including:

MediaDevices API: Used to request camera access.

Web Speech API: Used to convert translated text into speech.

Future versions can use REST APIs to connect the frontend to a backend and database

6.4 Authentication and Authorization

The current prototype provides the user interface for login and registration. Complete authentication and authorization require backend implementation.

A future implementation can use:

- Password hashing.
- JWT-based authentication.
- Role-based authorization.
- Input validation.
- Secure session management.

Possible roles include **User** and **Administrator**.

CHAPTER 7: FEATURES

7.1 List of Main Features

The major features of SignEase are:

1. Landing page.
2. User registration interface.
3. Login interface.
4. Dashboard.
5. Sign translator.
6. Camera access.
7. Demo sign recognition.
8. Text translation output.
9. Speech output.
10. Sign dictionary.
11. Translation history.
12. Favorite signs.
13. Learning module.
14. User profile.
15. Feedback interface.
16. Responsive user interface.

7.2 How Each Feature Works

Sign Translator

The user opens the translator module and interacts with the translation interface. The current prototype demonstrates translation using predefined/demo signs.

Camera Access

The application requests permission to access the user's camera through the browser. The camera provides the input interface required for future gesture-recognition functionality.

Text Output

After a demo sign is selected or recognized through the prototype workflow, the corresponding meaning is displayed as text.

Speech Output

The translated text can be converted into speech using the browser's Web Speech API.

Sign Dictionary

The dictionary contains signs and their meanings, allowing users to search or explore common signs.

History

Previous demo translations can be displayed in the history section. LocalStorage can be used to retain this information.

Favorites

Users can save frequently used signs and access them from the favorites section.

Learning Module

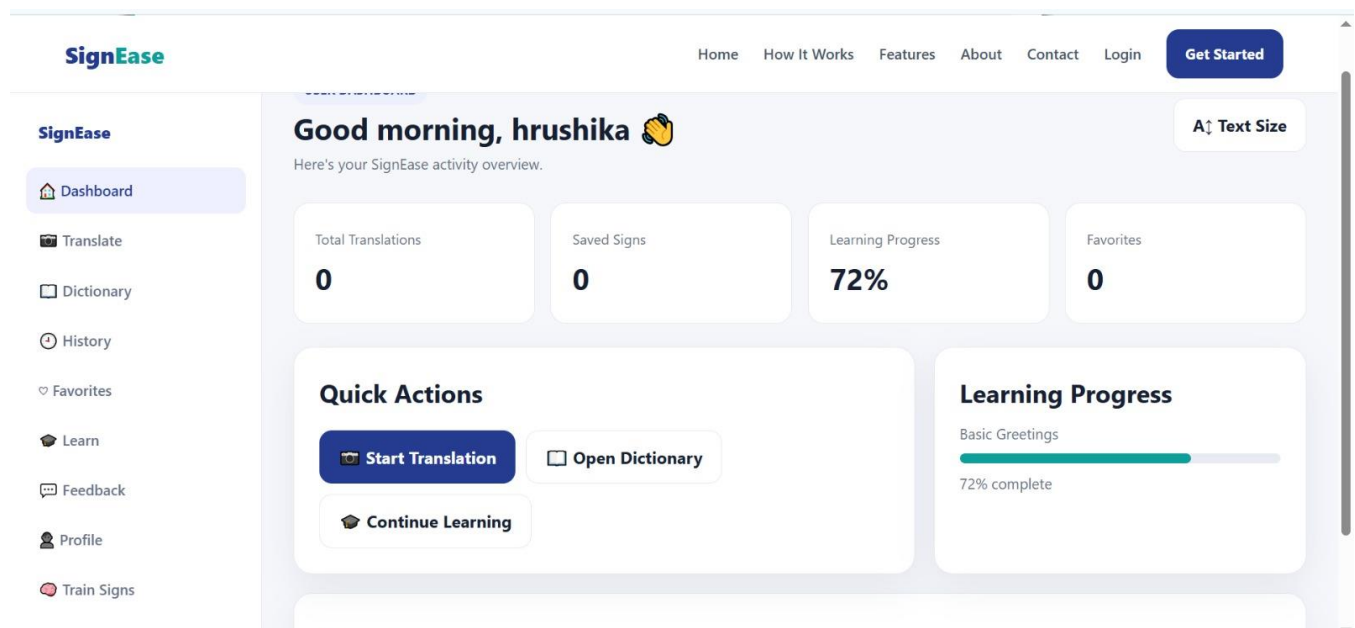
The learning section provides structured sign-language-related learning content.

Profile

The profile section allows users to view their profile-related information.

Feedback

Users can provide feedback and suggestions through the feedback interface.



SignEase

[Dashboard](#)[Translate](#)[Dictionary](#)[History](#)[Favorites](#)[Learn](#)[Feedback](#)[Profile](#)[Train Signs](#)

Live Camera

Start the camera to begin.

▶ Start Camera

■ Stop Camera

↻ Switch

Live ML Prediction

Start the camera to begin

Use this prediction !

Based on your captured training samples,
compared live using nearest-neighbor matching.

Demo Sign (manual)

Hello



Detected Sign

CHAPTER 8: TESTING

8.1 Functional Testing

Functional testing is performed to verify whether each module works according to its intended purpose.

The following areas should be tested:

- Page navigation.
- Buttons.
- Forms.
- Translator interface.
- Camera permission.
- Demo translation.
- Speech output.
- Dictionary.
- Favorites.
- History.
- Learning module.
- Profile.
- Feedback.

8.1 Integration Testing

Integration testing verifies whether different components work together correctly.

Examples include:

- Translator + camera access.
- Translator + text output.
- Text output + speech synthesis.
- Favorites + LocalStorage.
- History + LocalStorage.
- Dashboard + application modules.

8.2 Tools Used for Testing

- Postman — manual and scripted API testing
- curl (command line) — scripted regression testing across every phase of development
- MySQL command-line client — direct verification of table contents and constraint behavior

- Manual browser testing __ verifying the React frontend's behavior and appearance

8.3 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Open SignEase	Landing page loads successfully	Pass
2	Open Login	Login interface is displayed	Pass
3	Open Registration	Registration interface is displayed	Pass
4	Open Dashboard	Dashboard loads correctly	Pass
5	Open Translator	Translator interface appears	Pass
6	Request camera access	Browser asks for camera permission	Pass
7	Allow camera	Camera stream becomes available if supported	Pass

8	Select demo sign	Corresponding text is displayed	Pass
9	Click speech button	Translated text is spoken	Pass
10	Open dictionary	Sign information is displayed	Pass
11	Add favorite	Selected sign is saved	Pass
12	Open history	Previous demo translations are displayed	Pass
13	Refresh page	LocalStorage data remains where implemented	Pass
14	Open learning	Learning section is displayed	Pass
15	Open profile	Profile section is displayed	Pass
16	Submit feedback	Feedback interface accepts user input	Pass

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The current SignEase frontend can be deployed locally using Visual Studio Code.

Steps:

- 10 Install Visual Studio Code.
- 11 Open the SignEase project folder.
- 12 Open the main HTML file.
- 13 Install the Live Server extension if required.
- 14 Right-click the HTML file.
- 15 Select **Open with Live Server**.
- 16 The application opens in a browser.
- 17 Allow camera permission when testing the translator.
- 18 Test all modules.

•

18.2 Environment Configuration

The current prototype does not require a backend server or database configuration.

The main requirements are:

- Modern web browser.
- Camera permission.
- JavaScript enabled.
- Local development server where required.

Future backend implementation will require environment variables for database credentials, authentication secrets, API configuration, and other sensitive settings.

18.3 Hosting of Frontend and Backend

The current prototype can be hosted as a static web application.

Possible future deployment architecture:

Frontend → Web Hosting

Backend → Application Server

Database → Database Server

ML Model → ML/Inference Service

These components can communicate through secure APIs

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Several challenges were encountered or considered during the development of SignEase.

The first challenge was designing an interface that could contain multiple modules without making navigation complicated.

Another challenge was handling camera access through browser permissions. Different browsers and devices may behave differently when requesting camera access.

Speech output also depends on browser support and the available speech synthesis voices.

Another major challenge is real-time sign recognition. Accurate sign recognition requires image processing, hand detection, feature extraction, and a trained machine-learning model.

10.2 Solutions Applied

A modular interface was designed to organize the different sections of the application.

Browser APIs were used for camera access and speech synthesis instead of requiring external services for the basic prototype.

LocalStorage was used to demonstrate client-side persistence for history and favorites.

The translator was initially implemented using predefined/demo recognition so that the complete user interface and workflow could be demonstrated before integrating an ML model.

10.3 Current Limitations or Known Bugs

The current project has the following limitations:

- Real-time machine-learning sign recognition is not yet implemented.
- The translator currently uses predefined/demo recognition.
- There is no connected backend server.
- There is no centralized database in the current prototype.
- Login and registration are interface-level features unless connected to a backend authentication system.
- LocalStorage data is limited to the browser/device.
- Translation accuracy cannot be evaluated until a trained ML model is integrated.
- Camera functionality depends on browser permissions and device availability.
- Speech synthesis depends on browser support and available voices.
- Continuous sentence-level sign recognition is not currently implemented.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

Future versions of SignEase can include:

1. Real-time sign-language recognition.
2. Machine-learning-based gesture classification.
3. Computer vision-based hand tracking.
4. Continuous sign sequence recognition.
5. Sentence-level translation.
6. Larger sign vocabulary.
7. Support for multiple sign languages.
8. Improved sign dictionary.
9. Video-based sign tutorials.
10. User learning progress.
11. Secure user authentication.
12. Backend database integration.
13. Admin management system.
14. Cloud-based deployment.
15. Improved mobile support.

11.2 Possible Integrations or Optimizations

Computer vision frameworks such as OpenCV and MediaPipe can be integrated to detect and track hand movements.

Machine-learning frameworks such as TensorFlow can be used to develop a gesture-classification model.

A backend using Node.js and Express.js can manage users and application data.

MySQL or MongoDB can be integrated for persistent data storage.

Performance can be improved by processing camera frames efficiently and reducing unnecessary model predictions.

A cloud deployment can allow users to access the application from different devices.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

SignEase is a web-based Sign Language Translator application designed to support sign-language learning and communication. The application provides a centralized interface containing translation, dictionary, learning, history, favorites, profile, and feedback features.

The project demonstrates how HTML, CSS, JavaScript, browser APIs, and LocalStorage can be combined to create an accessibility-focused web application.

The current prototype provides the foundation for a more advanced sign-language recognition system.

12.2 What Was Achieved

The project achieved the development of a functional frontend prototype containing:

- SignEase landing page.
- Login and registration interfaces.
- Dashboard.
- Translator interface.
- Camera access.
- Demo translation functionality.
- Text output.
- Speech output.
- Sign dictionary.
- Translation history.
- Favorites.
- Learning section.
- Profile interface.
- Feedback interface.
- Responsive design.
- Browser-based data persistence.

The project also establishes a clear architecture for future integration with machine learning, backend

services, and a database.

12.3 Skills Learned

During the development of SignEase, the following skills were developed:

- HTML5 web development.
- CSS3 styling.
- JavaScript programming.
- Responsive web design.
- User interface design.
- Browser API integration.
- Camera access.
- Speech synthesis.
- LocalStorage.
- Database planning.
- ER diagram and schema design.
- Software testing.
- Project documentation.
- Understanding of machine-learning integration requirements.
- Accessibility-focused application development.

CHAPTER 13: REFERENCES

- ❑ **MDN Web Docs – HTML**
<https://developer.mozilla.org/en-US/docs/Web/HTML>
- ❑ **MDN Web Docs – CSS**
<https://developer.mozilla.org/en-US/docs/Web/CSS>
- ❑ **MDN Web Docs – JavaScript**
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- ❑ **MDN Web Docs – MediaDevices API**
<https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices>
- ❑ **MDN Web Docs – Web Speech API**
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- ❑ **MDN Web Docs – Web Storage API**
https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- ❑ **OpenCV Documentation**
<https://docs.opencv.org/>
- ❑ **MediaPipe Documentation**
<https://ai.google.dev/edge/mediapipe/solutions/guide>
- ❑ **TensorFlow Documentation**
<https://www.tensorflow.org/>
- ❑ **MySQL Documentation**
<https://dev.mysql.com/doc/>
- ❑ **Visual Studio Code Documentation**
<https://code.visualstudio.com/docs>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Camera Access

```
    async function startCamera() {
      try {
        const stream = await
        navigator.mediaDevices.getUserMedia({
          video: true
        });

        const video =
        document.getElementById("camera");
        video.srcObject = stream;
      } catch (error) {
        console.error("Camera access failed:",
        error);
      }
    }
  }
```

Speech Output:

```
function speakText(text) {
  if ("speechSynthesis" in window) {
    const speech = new SpeechSynthesisUtterance(text);
    window.speechSynthesis.speak(speech);
  }
}
```

LocalStorage:

```
function saveFavorite(sign) {
  let favorites =
    JSON.parse(localStorage.getItem("favorites")) || [];

  favorites.push(sign);

  localStorage.setItem(
    "favorites",
    JSON.stringify(favorites)
  );
}
```

Appendix B: Installation and Setup Instructions

Requirements

- Visual Studio Code.
- Modern web browser.
- Webcam for camera testing.
- Live Server extension.

Installation Steps

1. Install Visual Studio Code.
2. Create or open the SignEase project folder.

3. Add the project HTML, CSS, and JavaScript files.
4. Open the project in Visual Studio Code.
5. Install the Live Server extension.
6. Right-click the main HTML file.
7. Select **Open with Live Server**.
8. Open the generated local URL in the browser.
9. Allow camera permission when requested.
10. Test the different application modules.

Appendix C: Default Login Credentials

The current SignEase frontend prototype does not use a connected backend authentication system.

Therefore, there are no official database-backed default login credentials for the current version.

If a demo login has been configured in your actual HTML/JavaScript code, enter those exact credentials here.

Appendix D: API Reference Summary

The current prototype does not contain a custom REST API.

The following browser APIs are used:

API	Purpose
-----	---------

MediaDevices API	Provides access to the device camera
------------------	--------------------------------------

Web Speech API	Converts translated text into speech
----------------	--------------------------------------

LocalStorage API	Stores demo history and favorites
------------------	-----------------------------------

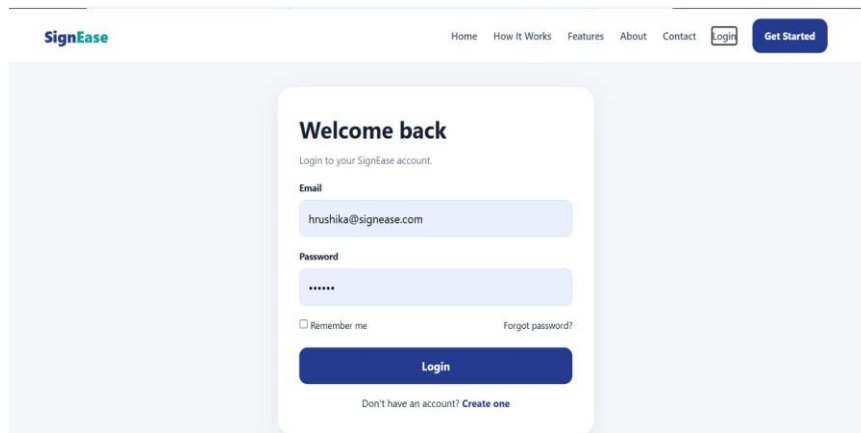
Future APIs can provide:

- User registration.
- User login.
- Sign management.
- Translation history.
- Favorites.
- Learning progress.
- Feedback.
- ML prediction requests.

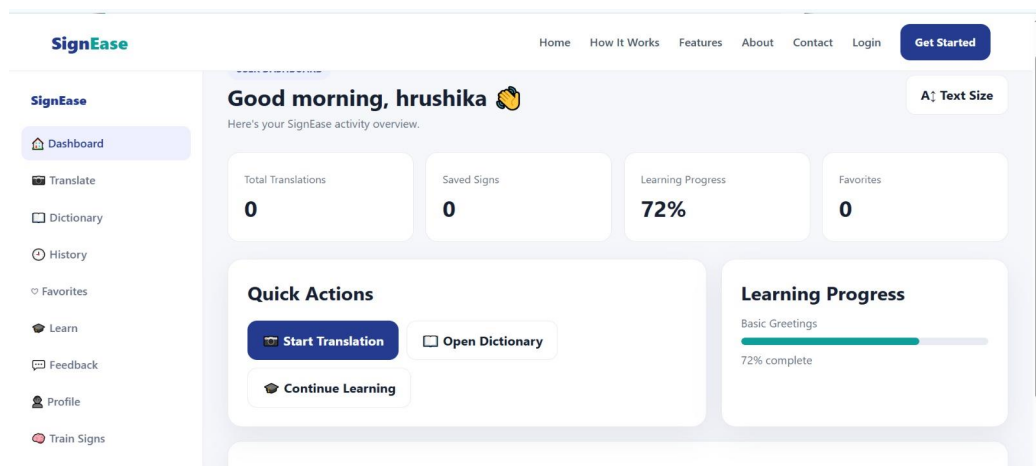
Appendix E: Screenshots

Suggested screenshots, in order:

1. Login Page

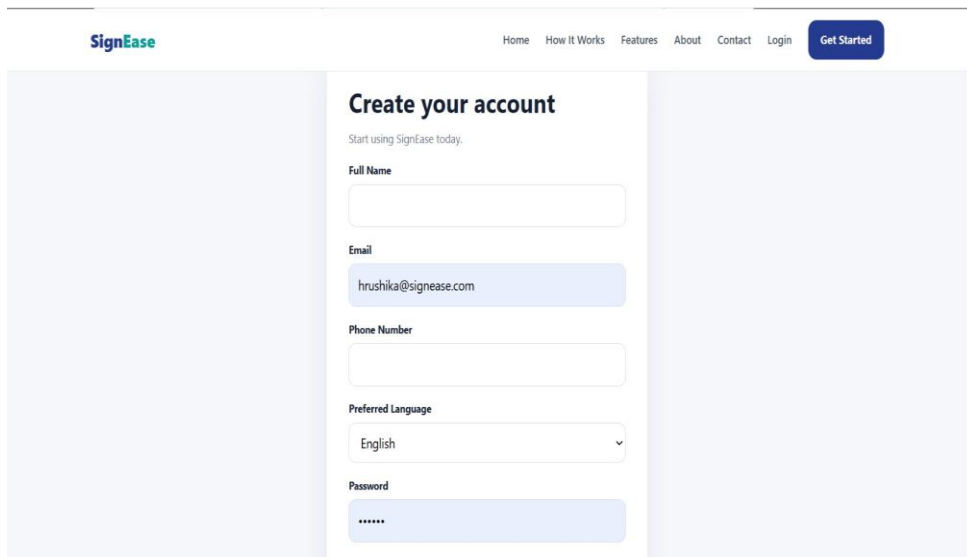


2. Dashboard



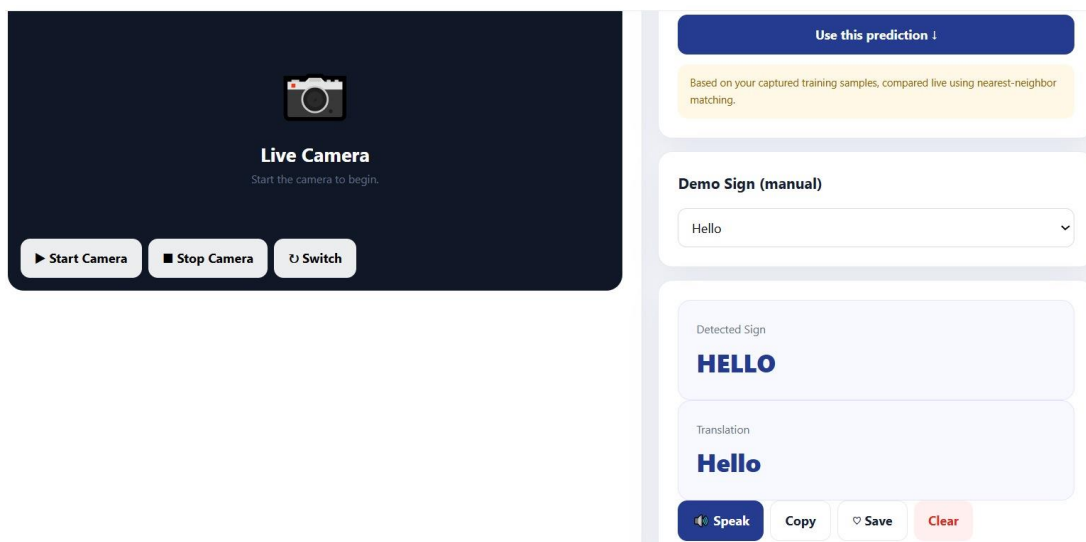
Events Page (student's view)

3. Registration



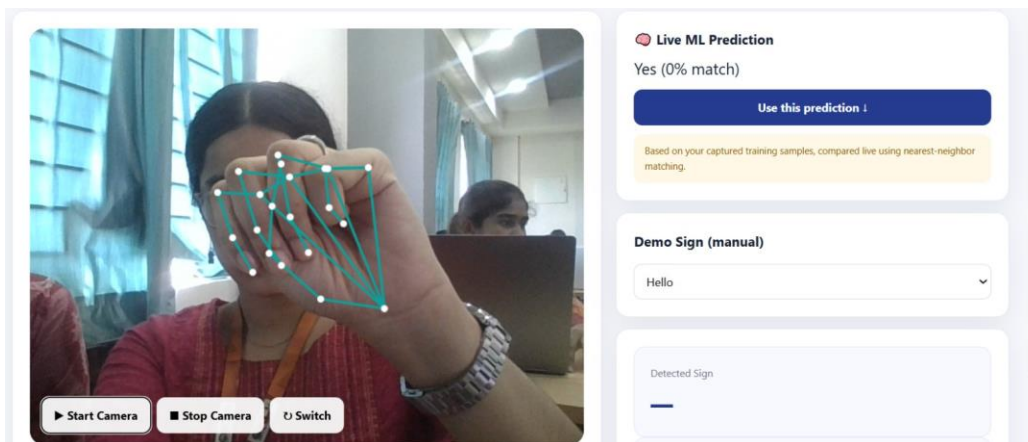
The registration form for SignEase is titled "Create your account" and includes the subtext "Start using SignEase today." The form fields are: Full Name (text input), Email (text input with "hrushika@signease.com" pre-filled), Phone Number (text input), Preferred Language (dropdown menu with "English" selected), and Password (password input with "*****" pre-filled). The navigation bar at the top includes links for Home, How It Works, Features, About, Contact, Login, and a "Get Started" button.

4. Translator



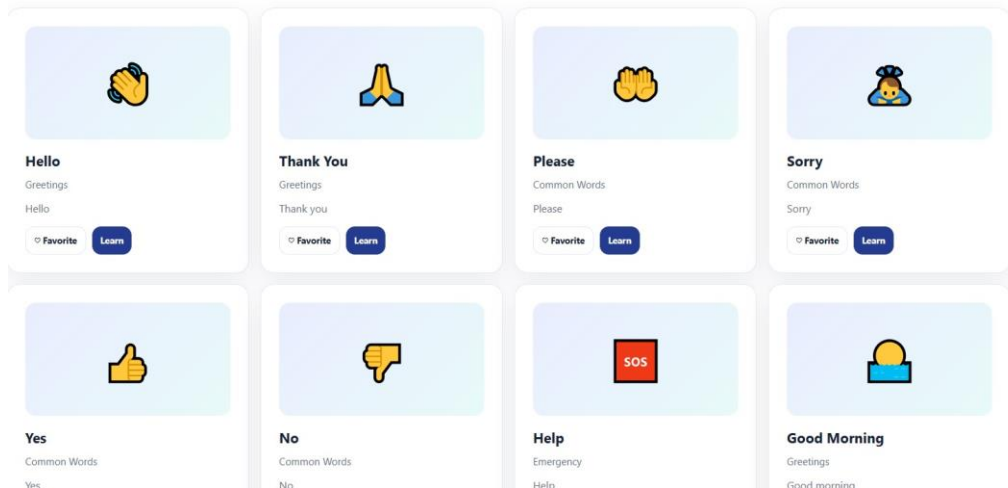
The translator interface is split into two main sections. The left section, titled "Live Camera", has the instruction "Start the camera to begin." and three buttons: "Start Camera", "Stop Camera", and "Switch". The right section, titled "Use this prediction !", contains a message: "Based on your captured training samples, compared live using nearest-neighbor matching." Below this is a "Demo Sign (manual)" dropdown menu with "Hello" selected. The "Detected Sign" section displays "HELLO" in large blue letters. The "Translation" section displays "Hello" in large blue letters. At the bottom of the right section are four buttons: "Speak", "Copy", "Save", and "Clear".

5. Translator Output



The translator output shows a live video feed of a person's hand making a sign. The hand is overlaid with a green wireframe model. The right section of the interface shows the "Live ML Prediction" results: "Yes (0% match)" and "Use this prediction !". Below this is the same "Demo Sign (manual)" dropdown menu with "Hello" selected. The "Detected Sign" section shows a blue line, indicating a prediction. The bottom of the interface has the same "Start Camera", "Stop Camera", and "Switch" buttons.

6. Sign Dictionary



7. Profile

PROFILE

Your Profile

hrushika
hrushika@signease.com

Account statistics

...

Edit Profile

Full Name
hrushika

Email
hrushika@signease.com

Phone
7788990077

Preferred Language
English

Save Changes

8. Feedback

FEEDBACK

Help us improve SignEase

Tell us about your experience.

Rating

☆☆☆☆☆

Feedback Type

Translation Accuracy

Message

Submit Feedback